

A Project On
Real-Time Data Ingestion and Processing for Word Count
Application



CENTRAL INSTITUTE OF TOOL DESIGN
HYDERABAD

BY
CHAGANTI VENKATESH,
MIDDELA AKSHITH
REDDY,

UNDER THE GUIDANCE OF
(S. VAMSI, M.Tech)
(Industry 4.0 Department)



**DEPARTMENT OF COMPUTER
ENGINEERING**

**BRILLIANT GRAMMAR SCHOOL
EDUCATIONAL SOCIETY GROUP OF
INSTITUTIONS INTEGRATED
CAMPUS, ABDULLAPUR (V), ABDULLAPURMET
(M), RANGAREDDY (D), HYDERABAD - 501 505.**



एमएसएमई-टूलरूम, हैदराबाद
MSME-TOOL ROOM, HYDERABAD
केंद्रीय उपकरण अभिकल्प संस्थान
Central Institute of Tool Design



BONAFIDE CERTIFICATE

This is to certify that this project report entitled “**Real-Time Data Ingestion and Processing for Word Count Application**” submitted to MSME TOOL ROOM(CITD), BALANAGAR, is a Bonafide record of work done by **CHAGANTI VENKATESH(22494-CS-044),MIDDELA AKSHITH REDDY(22494-CS-014),SURRAF NOMINI(22494-CS-009)** under my supervision from 18th November 2024 to 17th May 2025.

S. VAMSI

Industry 4.0 Engineer

(G. SANATHKUMAR)

Dy. Director (Trg.)

ACKNOWLEDGEMENT

We would like thank and honor our head of the department **G. SANATH KUMAR, DEPUTY DIRECTOR** who advised me throughout the completion of this work. I would like to extend my heartfelt thanks to my industry 4.0 instructor **S. VAMSI** for his continuous assistance, guidance, support, views, ideas encouragement and feedback without his help this project would have been a distant dream. We express heart-full thanks for the employees who contributed either directly or indirectly for the successful completion of industrial training. We thank each and every one of **MSME TOOL ROOM, HYDERABAD(CITD)** For their kind cooperation and all sorts of help bring out this industrial training Successfully.

CHAGANTI VENKATESH(22494-CS-044)
MIDDELA AKSHITH REDDY(22494-CS-014)
SURRAF NOMINI(22494-CS-009)

ABSTARCT

Word count is used to measure and analyze the frequency of words appearing in a text or a data stream. It is one of the most basic yet powerful techniques in text analysis, natural language processing (NLP), and big data applications. Understanding content trends becomes easier with word count, as it helps identify frequently mentioned words or topics across documents, chats, news feeds, and social media platforms. It also serves as the foundation for advanced analytics such as sentiment analysis, topic modeling, keyword extraction, and recommendation systems.

In real-time systems like social media or server logs, word count aids in spotting trending topics, identifying system issues, or monitoring customer concerns promptly. Search engines benefit from word frequency analysis to better rank and index content by determining keyword relevance. Additionally, it plays a crucial role in spam and fraud detection by identifying abnormal patterns in word usage, such as those found in fake reviews or scam messages. Businesses and content creators use it to optimize their material for SEO by managing keyword density. In IT systems, analyzing logs for words like "error," "failure," or "critical" can help detect technical issues early and automate alerts for quicker response.

This project focuses on building a real-time data processing pipeline using Apache Kafka and Apache Spark to perform word count analytics on streaming data. In today's digital world, real-time insights are crucial for many industries. By leveraging Kafka as the data ingestion platform and Spark for real-time data processing, this project demonstrates an efficient and scalable architecture for streaming applications. The work involves setting up a local development environment using Windows Subsystem for Linux (WSL), installing essential tools like Java, Python, and Spark, and developing a Python-based word count program that reads streaming data from a Kafka topic and processes it in real time. This project highlights the power of real-time big data frameworks and provides a fundamental implementation model for future real-world streaming solutions.

CONTENT

BONFIDE3

ACKNOWLEDGEMENT.....4

ABSTRACT.....5

LIST OF FIGURES.....8,9

PROJECT

1.INTRODUCTION(11-14)

1.1 Introduction to Real-Time data Analysis in big data.....11

1.2 Introduction to Apache Spark.....12

1.3 Introduction to Apache Kafka.....13

1.4 Introduction to Windows Sub-System for Linux(Ubuntu).....14

2.INSTALLATION OF WSL,SPARK,KAFKA(14-20)

2.1 Installing WSl (windows sub-system for Linux).....15

2.2 Installing java -jdk 11.....15

2.3 Installing python 3.8 and pySpark.....16

2.4 Installing Apache spark 3.5.116

2.5 Installing Apache Kafka 3.7.018

2.6 Setting path for installed applications18

2.7 Installing spark-Kafka connector19

3.WORD-COUT PROGRAM FOR TESTING(20-22)

3.1 Writing the program and saving.20

3.2 Starting zookeeper and server.21

3.3 Starting Kafka producer and consumer21

3.4 Running the python code22

4. RESULT23

5.CONCULSION & FUTURE SCOPE24

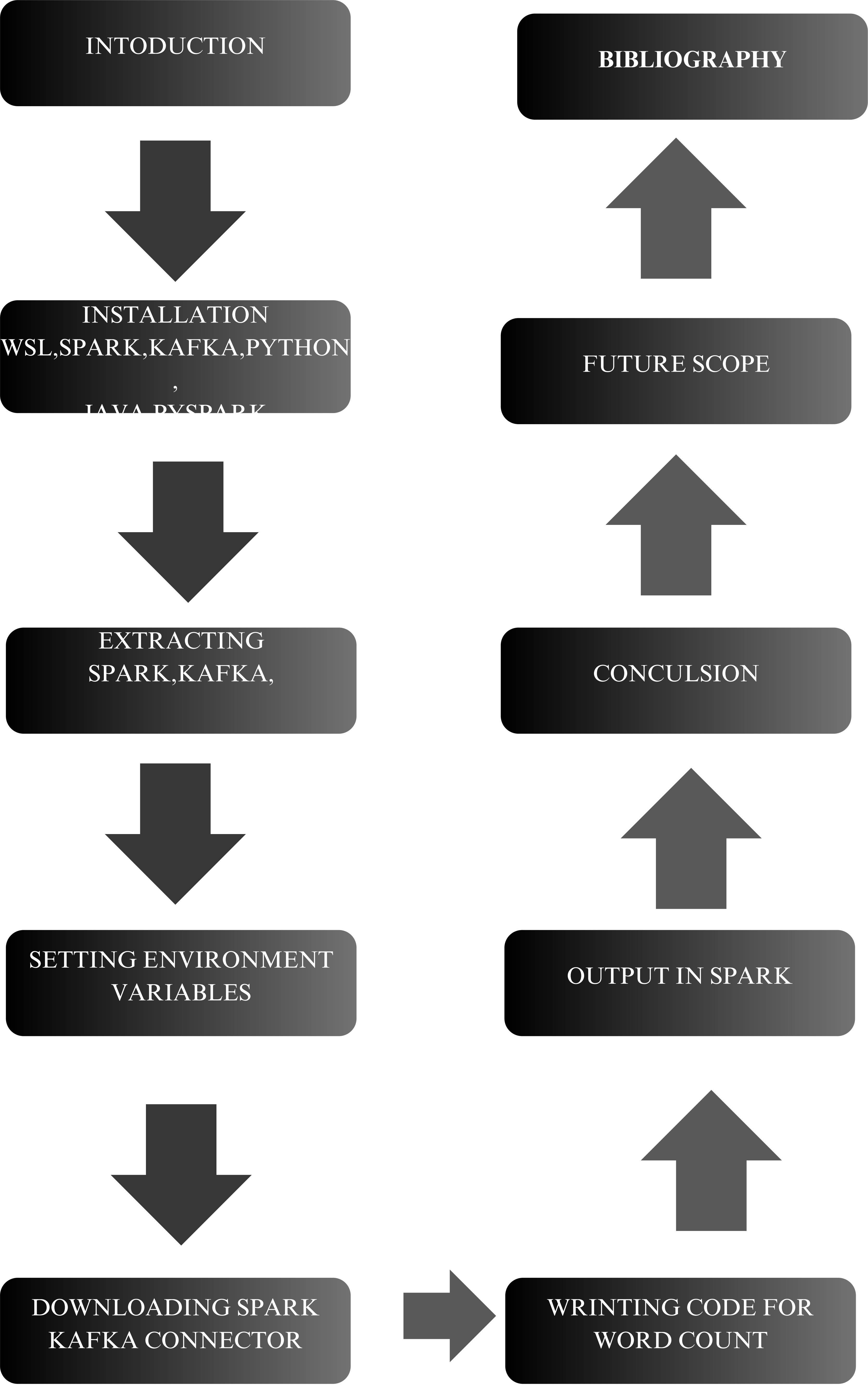
BIBLIOGRAPHY.....27

LIST OF FIGURES

Figure No.	Title / Description	Context / Section	Page No.
Fig 1.1	REAL TIME DATA PROCESSING	REAL TIME DATA PROCESSING	11
Fig 1.2	APACHE SPARK	APACHE SPARK	12
Fig 1.3	APACHE KAFKA	APACHE KAFKA	13
Fig 1.4	WSL	WSL	14
Fig 2.1	WSL Installation Command	Installing WSL on Windows	15
Fig 2.2	Updating Packages in Ubuntu	Installing Java JDK 11	15
Fig 2.3	Installing OpenJDK	Installing Java JDK 11	16
Fig 2.4	Checking Java Version	Installing Java JDK 11	16
Fig 2.5	Java Version Output	Installing Java JDK 11	16
Fig 2.6	Software Dependencies for Python	Installing Python 3.8 and PySpark	16
Fig 2.7	Adding Dead Snakes PPA	Installing Python 3.8	17
Fig 2.8	Installing Python 3.8	Installing Python 3.8	17
Fig 2.9	Setting Python 3.8 as Default	Installing Python 3.8	17
Fig 2.10	Checking Python Version	Installing Python 3.8	17
Fig 2.11	Setting up pip	Installing PySpark	17
Fig 2.12	Installing PySpark via pip	Installing PySpark	17
Fig 2.13	Downloading Spark	Installing Apache Spark 3.5.1	18
Fig 2.14	Extracting Spark	Installing Apache Spark 3.5.1	18
Fig 2.15	Downloading Kafka	Installing Apache Kafka 3.7.0	18
Fig 2.16	Extracting Kafka	Installing Apache Kafka 3.7.0	18

Fig 2.17	Editing Environment Variables	Setting Environment Variables	18
Fig 2.18	Setting JAVA/SPARK/KAFKA Path	Setting Environment Variables	19
Fig 2.19	Using Spark-Kafka Connector via --packages	Installing Spark-Kafka Connector	19
Fig 2.20	Manual Download of JAR Connector	Installing Spark-Kafka Connector	19
Fig 2.21	Manual Download of JAR Connector 2	Installing Spark-Kafka Connector	19
Fig 2.22	Placing JARs in Spark Jars Folder	Installing Spark-Kafka Connector	19
Fig 3.1	Interface (pyspark shell)	spark interface	20
Fig 3.2	word -count program	Writing Word Count Program	21
Fig 3.3	starting zookeeper	starting zookeeper	21
Fig 3.4	starting server	starting server	21
Fig 3.5	creating a topic	creating a topic	22
Fig 3.6	starting a producer	Kafka Producer/Consumer Setup	22
Fig 3.7	starting a consumer	starting a consumer	22
Fig 3.8	checking	Word Count Test Output	22
Fig 3.9	running code	Word Count Test Output	22
Fig 3.10	producer data	producer data	22
Fig 4.1	Spark Output Showing Connection Success	out put	23
Fig 4.2	file solution	file solution	23
Fig 6.1	sensor code	sensor code	25
Fig 6.2	processing code	processing code	26
Fig 6.3	result	result	26

PROJECT ARCHITECTURE



CHAPTER- INTRODUCTION

1.1 Introduction to Real-Time data Analysis in big data

In the age of digital transformation, organizations are generating massive volumes of data from various sources like social media, IoT devices, mobile apps, sensors, and transactions. While traditional batch processing is useful for analyzing large datasets over time, there's an increasing demand for real-time insights. This is where real-time data processing comes into play.



Fig 1.1 REAL TIME DATA PROCESSING

• WHAT IS REAL-TIME DATA PROCESSING?

Real-time data processing refers to the continuous input, processing, and output of data with minimal latency. It allows businesses to analyze and act on data as soon as it is generated, enabling faster decision-making.

Key Characteristics:

1. **Low latency:** Near-instant processing (often in milliseconds or seconds).
2. **Continuous data flow:** Handles streaming data, unlike batch systems which wait for a full dataset.
3. **Immediate response:** Triggers alerts, actions, or decisions in real-time.

• Why Real-Time Processing is Important in Big Data ?

In big data environments, the ability to handle velocity (the speed of data) is critical.

Real-time processing enables:

1. **Fraud detection** (e.g., in financial transactions)
2. **Monitoring systems** (e.g., network traffic, health monitors)
3. **Customer behavior tracking** (e.g., personalized offers, recommendation engines)
4. **Operational intelligence** (e.g., supply chain tracking, logistics)

1.2 Introduction to Apache Spark

Traditional data processing tools like Hadoop MapReduce are powerful but can be slow, especially for iterative or real-time tasks. Apache Spark was built to address these limitations, offering:

- 1.Speed:** In-memory computing makes Spark up to 100x faster than Hadoop MapReduce for certain tasks.
- 2.Ease of use:** High-level APIs in Scala, Java, Python (PySpark), and R.
- 3.Versatility:** Supports batch processing, real-time streaming, machine learning, and graph processing—all in one framework.



Fig 1.2 APACHE SPARK

• How Spark

Works

Spark runs in a clustered environment, managing tasks across distributed systems. It uses a concept called RDD (Resilient Distributed Dataset), a fault-tolerant collection of elements that can be operated on in parallel.

• Key

- 1.Driver:** The process that runs the main function and creates SparkContext.
- 2.Executor:** Distributed processes that run tasks and return results.
- 3.Cluster Manager:** (e.g., YARN, Mesos, Kubernetes, or Spark's standalone cluster manager) handles resource allocation.

• Core Components of Apache Spark

1.Spark Core

The foundation of the system, responsible for basic I/O, task scheduling, and memory management.

2.Spark SQL

Enables querying structured data with SQL or the DataFrame API.

3.Spark Streaming

Allows real-time data stream processing using micro-batches.

4.MLlib (Machine Learning Library)

Provides scalable machine learning algorithms like classification, regression, clustering, etc.

5.GraphX

API for graph processing and graph-parallel computation.

• Use Cases of Apache Spark

- 1.Data warehouse modernization
- 2.Real-time data analytics
- 3.Machine learning model training at scale
- 4.ETL (Extract, Transform, Load) pipelines
- 5.Log and event processing

1.3 Introduction to Apache Kafka

Apache Kafka is an open-source, distributed event streaming platform used to build real-time data pipelines and streaming applications. Originally developed at LinkedIn and later donated to the Apache Software Foundation, Kafka has become the backbone of many modern data architectures.

• What is Apache Kafka?

At its core, Kafka is a high-throughput, fault-tolerant system for handling real-time data feeds. It enables systems to publish, subscribe to, store, and process streams of records in real time.

• Kafka is commonly used for:

1. Messaging
2. Website activity tracking
3. Log aggregation
4. Stream processing
5. Event sourcing
6. Real-time analytics

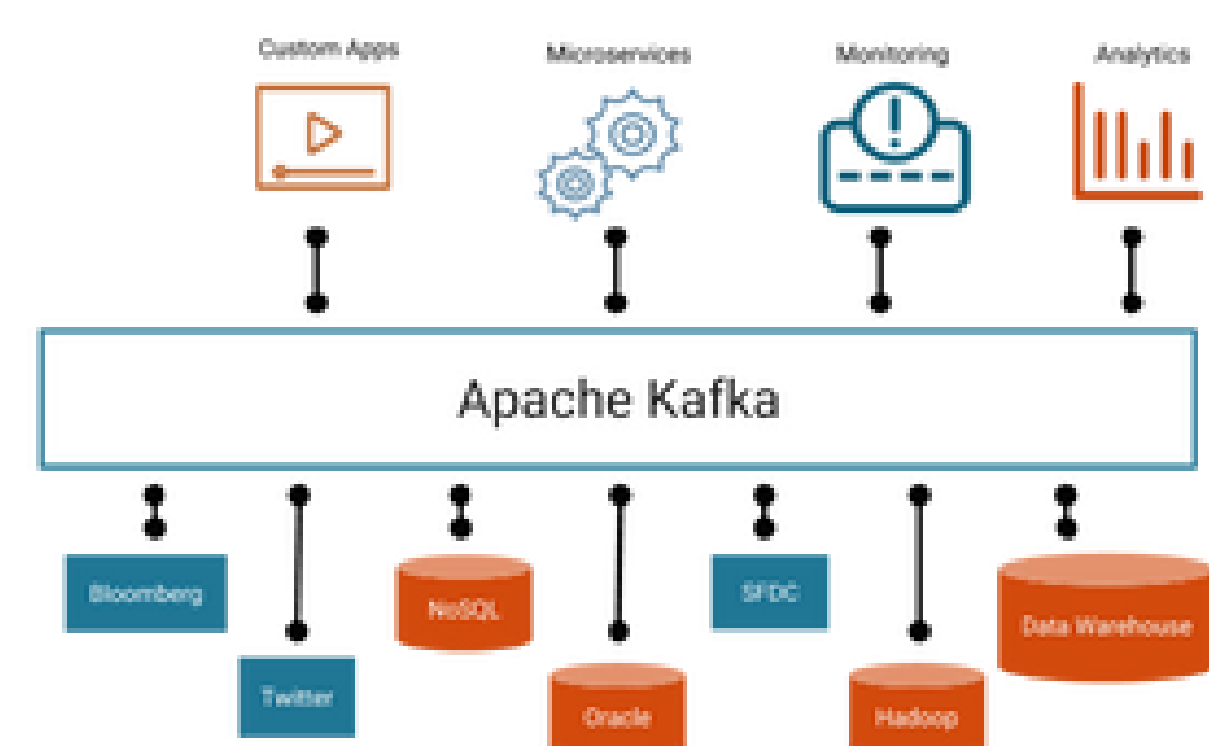


Fig 1.3 APACHE

• Key Concepts in Apache Kafka:

1.Producer:

Sends (publishes) data (messages) to Kafka topics.

2.Consumer:

Reads (subscribes to) data from Kafka topics.

3.Topics:

Categories or channels to which records are sent and from which consumers read.

4.Brokers:

Kafka servers that store and serve data.

5.Partitions:

Each topic is split into partitions, enabling Kafka to scale horizontally and process messages in parallel.

6.ZooKeeper:

Manages Kafka's cluster metadata (though newer versions of Kafka can now run without ZooKeeper).

- **Why Use Apache Kafka?**

1. **High throughput:** Capable of handling millions of messages per second.
2. **Scalability:** Easily scales across many servers and datacenters.
3. **Durability:** Messages are stored on disk and replicated for fault tolerance.
4. **Low latency:** Real-time processing with minimal delay.
5. **Fault-tolerant:** Handles node failures without losing data.

- **Real-World Use Cases:**

1. **LinkedIn:** Activity data tracking
2. **Uber:** Real-time event processing
3. **Netflix:** Monitoring and real-time analytics
4. **Airbnb:** Metrics collection and anomaly detection

1.4 Introduction to Windows Subsystem for Linux (WSL) – Ubuntu on Windows

- **Windows Subsystem for Linux (WSL)** is a powerful feature in Windows that allows you to run a Linux environment directly on Windows, without the need for a virtual machine or dual-boot setup. It's particularly popular with developers, sysadmins, and data scientists who want the flexibility of Linux tools on a Windows machine.

- **What is WSL?**

WSL enables Windows users to run a full Linux kernel and tools alongside their usual Windows apps. It's tightly integrated with the Windows system, making it fast, lightweight, and super convenient. When you install Ubuntu via WSL, you get access to a fully functional Ubuntu terminal, where you can:

1. Run Linux command-line tools (like bash, grep, awk, sed)
2. Install packages using apt
3. Use Python, Git, Node.js, Ruby, and more
4. Work with Docker (with WSL 2)
5. Manage code, build apps, and run servers

- **Advantages of**

WSL

1. No need for a virtual machine
2. Lightweight and fast
3. Deep integration with Windows (file system, clipboard, etc.)
4. Great for web and software development
5. Free and easy to set up



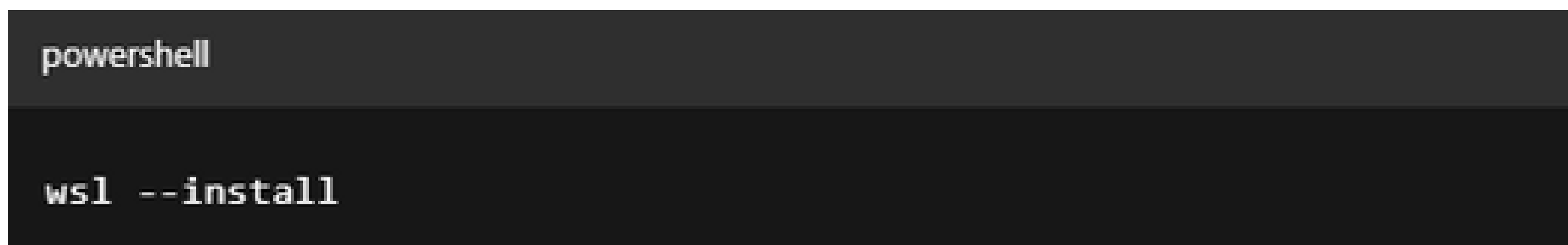
Fig 1.4

CHAPTER-2

INSTALLATION OF WSL, SPARK , KAFKA

2.1 installing WSl(windows sub-system for linux)

- **Step 1: Open PowerShell as Administrator**
 1. Click Start, type PowerShell
 2. Right-click Windows PowerShell
 3. Choose Run as Administrator
- **Step 2: Install WSL (One Command Install)**



```
powershell
wsl --install
```

Fig 2.1 wsl installation command

This command will:

1. Enable required Windows features
 2. Download and install Ubuntu by default
 3. Set WSL 2 as the default version
 4. ⚠ If you see a message asking to restart your computer — do it.
- **Step 3: Set Up Ubuntu (First Time Only)**

After installation and reboot:

1. Search for "Ubuntu" in the Start Menu
2. Open it — it will finalize setup
3. Create a username and password for your Ubuntu session
4. That's it! You now have a fully functional Linux terminal on your Windows system.

2.2 insatalling java -jdk 11

Java 11 is a Long-Term Support (LTS) version — perfect for most dev work.

Works for: Ubuntu 20.04, 22.04, and newer versions.

- **Step 1: Update Package**

List
Open your Ubuntu terminal and run:

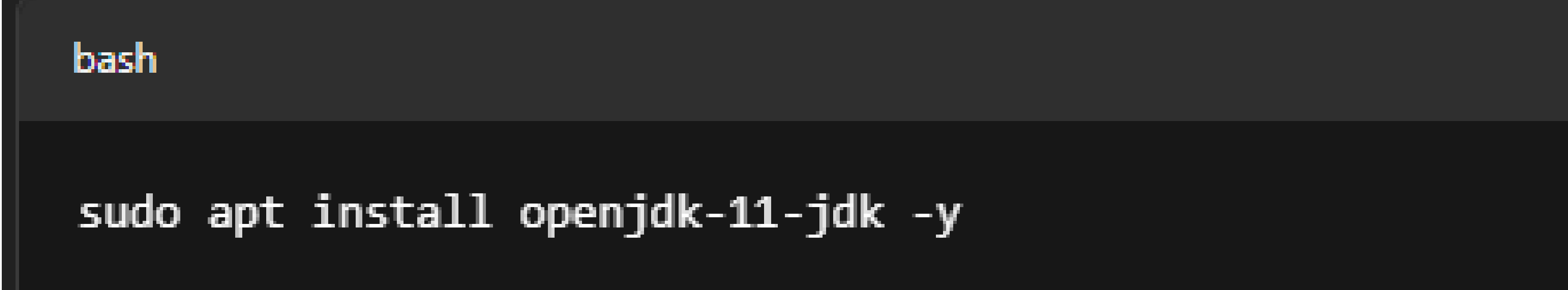


```
bash
sudo apt update
```

Fig 2.2 updating packages

- **Step 2: Install OpenJDK**

11



```
bash
```

```
sudo apt install openjdk-11-jdk -y
```

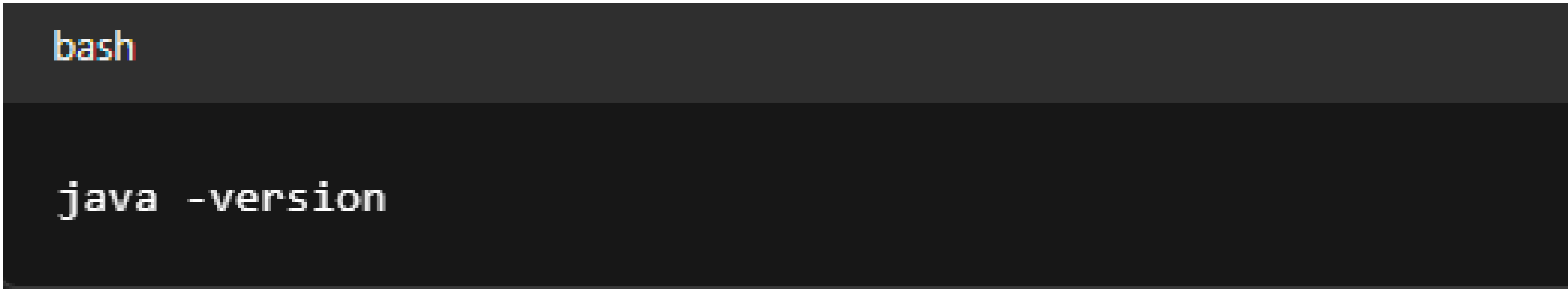
Fig 2.3 installing open JDk

This installs the Java Development Kit (JDK), which includes:

1. Java compiler (javac)
2. Java Runtime (java)
3. Other essential dev tools

- **Step 3: Verify Installation**

After installation completes, check the Java version:



```
bash
```

```
java -version
```

Fig 2.4 checking version

You should see output like:

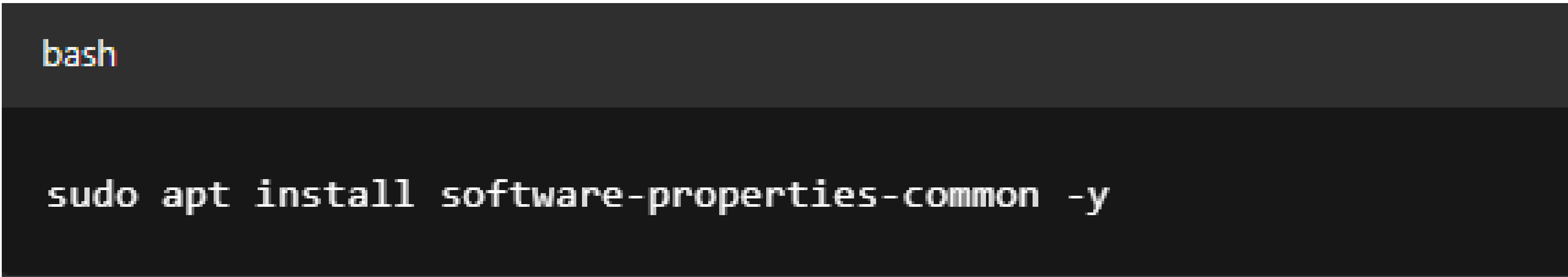


```
openjdk version "11.0.x" ...
```

Fig 2.5 version output

2.3 installing python 3.8 and pyspark

Install required dependencies:



```
bash
```

```
sudo apt install software-properties-common -y
```

Fig 2.6 software dependency

- **What is software-properties-common?**

It's a package that provides utilities for managing software repositories via the command line.

- **Add the deadsnakes PPA (contains older Python versions):**

```
bash

sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
```

Fig 2.7 dead snakes

- **Install Python 3.8:**

```
bash

sudo apt install python3.8 -y
```

Fig 2.8 installing python 3.8

- **Set Python 3.8 as Default:**

```
bash Copy

sudo update-alternatives --install /usr/bin/python3 python3 /usr/bin/python3.8 1
```

Fig 2.9 setting default

You can check the version:

```
bash

python3 --version
```

Fig 2.10 version check

- **Set Up pip for Python 3.8**

```
bash

sudo apt install python3.8-venv python3.8-distutils -y
curl -sS https://bootstrap.pypa.io/get-pip.py | sudo python3.8
```

Fig 2.11 setting up pip

- **Install PySpark via
pip:**

```
bash

python3.8 -m pip install pyspark
```

Fig 2.12 pyspark

• 2.4 installing Apache spark 3.5.1

run this command in ubuntu terminal to download spark 3.5.1

```
wget https://archive.apache.org/dist/spark/spark-3.5.1/spark-3.5.1-bin-hadoop3.tgz
```

Fig 2.13 downloading spark

1. this command download original spark 3.5.1 from archive as it is older version.
2. we use this version for better compatibility b/w spark and kafka.

Then extract it:

```
bash

tar -xvzf spark-3.5.1-bin-hadoop3.tgz
```

Fig 2.14 extracting

• 2.5 installing Apache kafka 3.7.0

1. Works on Ubuntu 20.04, 22.04, or WSL
 2. 🛠 Assumes Java 11 is already installed
 3. 🌿 Kafka 3.7.0 requires Java 11+
- open terminal and enter following command to download kafka

```
wget https://archive.apache.org/dist/kafka/3.7.0/kafka_2.13-3.7.0.tgz
```

Fig 2.15 downloading kafka

- to extract kafka run following command:

```
tar -xzf kafka_2.13-3.7.0.tgz
mv kafka_2.13-3.7.0 ~/kafka
```

Fig 2.16 extracting kafka

• 2.6 setting path for installed applications

1. first run this command to edit environment variables:

```
nano ~/.bashrc
```

Fig 2.17 To edit environment variables

2. now we have to set environment variable for java ,spark ,kafka as shown in the fig we have to add this lines to set path.

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export PATH=$PATH:$JAVA_HOME/bin
nano ~/.bashrc|
export KAFKA_HOME=~/.kafka
export PATH=$PATH:$KAFKA_HOME/bin
export SPARK_HOME=~/.spark
export PATH=$PATH:$SPARK_HOME/bin
```

Fig 2.18 setting path

• 2.7 installing spark-kafka connector

1. You don't need to manually install a JAR. Spark can fetch it automatically with the --packages option.
2. For Spark 3.5.1 + Kafka:

```
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.13:3.5.1 your_script.py
```

Fig 2.19 spark-kafka connector

3.this will only work when your connected to internet .

- If you want to run Spark offline (or not use the --packages feature), download the connector JAR manually:

```
~$ wget https://repol.maven.org/maven2/org/apache/spark/spark-sql-kafka-0-10_2.13/3.5.1/spark-sql-kafka-0-10_2.13-3.5.1.jar
```

Fig 2.20. manual download

- Also download Kafka client JAR (required):

```
~$ wget https://repol.maven.org/maven2/org/apache/kafka/kafka-clients/3.7.0/kafka-clients-3.7.0.jar
```

Fig 2.21 manual download 2

- Place both JARs in a folder (e.g., ~/spark-jars) and start Spark with:

```
:~$ pyspark --jars ~/spark-jars/spark-sql-kafka-0-10_2.13-3.5.1.jar,~/spark-jars/kafka-clients-3.7.0.jar
```

Fig 2.22 placing jar files

• Important things to be checked

1. Spark and Kafka versions must match connector compatibility (which we ensured above).
2. Kafka must be running when connecting.
3. For writeStream (sending data to Kafka), you need to provide topic, checkpointLocation, etc

CHAPTER-3

3.WORD-COUNT PROGRAM FOR TESTING

- **3.1 Witing the program and saving**

1.make sure that your spark is working to check type this command:

spark-shell or pyspark

2.If spark is working it will show output like this:

```
venkatesh@venky:~$ spark-shell
25/04/19 10:24:08 WARN Utils: Your hostname, venky resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
25/04/19 10:24:08 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/04/19 10:24:12 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Spark context Web UI available at http://10.255.255.254:4040
Spark context available as 'sc' (master = local[*], app id = local-1745058253352).
Spark session available as 'spark'.
Welcome to

  ____      _
 / ___|  __| | | |
 \___ \  | | | | | |
  ___) | | | | | | |
 |_____|_|_|_|_|_|_|

version 3.5.1

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 11.0.26)
Type in expressions to have them evaluated.
Type :help for more information.

scala> |
```

Fig 3.1 spark interface

- **possible**

error:

1.some times when you use pyspark or spark-shell command you may encounter problem like “no internal or external command found **spark**”.

2.This error occur due to not setting proper path in environment variables or not extracting spark or not downloading spark in wsl(Ubuntu) terminal.

- **solutio**

n:

1.download spark though ubuntu terminal using command which you can find in 2.4 topic.

2.set environment variable as shown in topic 2.6

3.this will solve the above error

- **After solving move to the python code**

- **Now open note pad and save this code as kafka_word_count at ubuntu home/user_name.**

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split

# Initialize SparkSession
spark = SparkSession.builder \
    .appName("KafkaWordCount") \
    .getOrCreate()

# Read streaming data from Kafka
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "test-topic") \
    .load()

# Convert the 'value' column from binary to string and split into words
words = df.select(
    explode(split(df.value.cast("string"), " ")).alias("word")
)

# Count occurrences of each word
word_counts = words.groupBy("word").count()

# Write the word counts to the console
query = word_counts.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()

# Keep the stream running
query.awaitTermination()

```

Fig 3.2 word count code

1. This python code reads data from test-topic which is in your kafka localhost:9092
2. Before executing this code we have to start kafka zookeeper, server and create a topic named test-topic and start producer and consumer so let's do that first.

◦ 3.2 starting zookeeper and server

1. To start zookeeper and sever use following code:

for zookeeper:

```
$ $KAFKA_HOME/bin/zookeeper-server-start.sh $KAFKA_HOME/config/zookeeper.properties &
```

Fig 3.3 starting zookeeper

for server:

```
~$ $KAFKA_HOME/bin/kafka-server-start.sh $KAFKA_HOME/config/server.properties &
```

Fig 3.4 starting server

• expected

error:

1. No command found zookeeper.start and server start

• soluti

on:

1. set path of kafka shown in topic 2.6 that should do it.

• 3.3 starting producer and consumer

1. First we have to create a topic to start producer and consumer let do it first.

- **To create topic use following command:**

```
$KAFKA_HOME/bin/kafka-topics.sh --create --topic test-topic --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
```

Fig 3.5 creating a topic

1. Using this command you can create a topic named test-topic.
2. here i used name test-topic you can give a name according to you .

- **Now we have to start producer and consumer**

1. To start producer use the following code:

```
$KAFKA_HOME/bin/kafka-console-producer.sh --topic test-topic --bootstrap-server localhost:9092
```

Fig 3.6 starting a producer

2. To start consumer use the following code:

```
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic my-topic --from-beginning
```

Fig 3.7 starting a consumer

expected errors:

1. topic not found when starting producer or consumer.

solution:

1. Make sure zookeeper and sevr are running.
2. Make sure your topic is created or using correct name while starting producer or consumer you can check if the topic is created or not by using the following **command:**

```
$KAFKA_HOME/bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

Fig 3.8 checking

3. This will show if your topic is present or not if not create it and try again.
4. Now we have started zookeeper , server , producer , consumer and checked if spark is working is or not and main important thing is the spark-kafka connector make sure you downloaded it and kept in jar folder of spark.

• **3.4 Now we can run the python code which we created to count words**

- open ubuntu terminal and use this command to run the code:

```
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 /home/venkatesh/kafka_word_count.py
```

Fig 3.9 running code

- After running the code open producer terminal typing anything the spark should read data from it and count the words.
- This is the data that i gave in producer terminal twice:

```
Once, there was a hare who was best friends with a tortoise. The hare was very proud of how fast he could run, so one day, he challenged the tortoise to a race. The tortoise agreed, even though everyone thought he was way too slow to win. The race began, and the hare raced so fast that he was far ahead of the tortoise.
```

Fig 3.10 producer data

CHAPTER-4

RESULT

- This is the output in the spark:

```
+-----+-----+
| word | count |
+-----+-----+
| slow | 2 |
| even | 2 |
| ahead | 2 |
| Moral | 1 |
| agreed, | 2 |
| could | 2 |
| are-what | 1 |
| who | 2 |
| you | 1 |
| steady | 1 |
| there | 2 |
| can | 1 |
| was | 10 |
| win. | 2 |
| how | 3 |
| friends | 2 |
| hare | 6 |
| too | 3 |
| one | 2 |
| giving | 1 |
+-----+-----+
only showing top 20 rows
```

Fig 4.1 output

1. This means our connection is success spark can read data from kafka topic

- **expected errors:**

1. File not found.
2. java.lang.NoClassDefFoundError: org/apache/kafka/common/serialization/ByteArraySerializer.

- **Solutio**

1. Specify your file path as shown while running the code like i did above

2. Next java.lang.NoClassDefFoundError; this error occurs due to version mismatch of spark-kafka connector or the file is corrupted or spark can't find or access it to avoid this we use spark-kafka package as shown above while running the code this will solve the error and we can see the output.

- Even after this it still show same error The JAR is corrupted. Clear the Ivy cache and re download it like this:

```
venkatesh@venky:~$ rm -rf /home/venkatesh/.ivy2/cache/*
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 /home/venkatesh/kafka_word_count.py
```

Fig 4.2 file solution

- That's it now your code should work with out any trouble

CHAPTER-5

CONCLUSION & FUTURE SCOPE

5.1 CONCLUSION

- The project successfully demonstrated real-time word count processing using Apache Kafka and Apache Spark. Setting up the environment on Windows through WSL and configuring the necessary components like Java, Python, and Spark-Kafka connectors ensured seamless integration. Kafka enabled efficient data ingestion, while Spark provided fast and reliable real-time data analysis. The Python-based implementation read incoming messages from a Kafka topic and processed them in real time to produce word count results. Challenges such as environment setup errors, dependency management, and connector compatibility issues were effectively handled. Overall, the project achieved its goal of demonstrating a working real-time data streaming system with

5.2 Future Scope

- **Enhanced Analytics:** Extend the system to perform complex real-time analytics like sentiment analysis, anomaly detection, or trend prediction.
- **Visualization Integration:** Implement real-time dashboards using tools like Apache Superset or Grafana for live monitoring of word count and data flow.
- **Scalability Testing:** Scale the Kafka-Spark system across multiple nodes to test its performance under high-load conditions.
- **Fault Tolerance and Recovery:** Add more advanced mechanisms to handle failures and ensure high availability of the streaming system.
- **Data Storage Integration:** Save the processed streaming results into distributed databases like Cassandra, MongoDB, or HDFS for further analysis.
- **Cloud Deployment:** Deploy the setup on cloud platforms like AWS EC2, Azure VMs, or GCP Compute Engine for broader accessibility and scalability.

◦ REAL-TIME SENSOR DATA STREAMING USING RASPBERRY PI, KAFKA, SPARK & TELISCALE.

we can also do a project for analysis of raspberry pi sensor data with the help of spark and kafka by connecting raspberry pi and WSL on your system with Tailscale a VPN. The sensor data is read and sent to kafka topic through this program :

```
import Adafruit_DHT
from kafka import KafkaProducer
import json
import time

sensor = Adafruit_DHT.DHT11
pin = 4 # GPIO pin connected to DHT sensor

producer = KafkaProducer(
    bootstrap_servers='100.x.x.x:9092', # Windows Tailscale IP
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

while True:
    humidity, temperature = Adafruit_DHT.read(sensor, pin)
    if humidity is not None and temperature is not None:
        message = {
            'temperature': temperature,
            'humidity': humidity,
            'timestamp': time.time()
        }
        producer.send('sensor-data', value=message)
        print("Sent:", message)
    else:
        print("Sensor read failed.")
    time.sleep(5)
```

Fig 6.1 sensor code

1. you have to first create a topic with name sensor-data..
2. Make sure to start your kafka zookeeper, server, producer and consumer before you run above program in raspberry pi.
3. you can see sensor data streaming into your kafka broker after running the code.

4. Now for processing the data from kafka using spark run this program on wsl:

```
kafka_spark_stream ✓

# kafka_spark_stream.py

from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StringType, FloatType

# Define schema for your sensor data
schema = StructType().add("temperature", FloatType()).add("humidity", FloatType())

# Create SparkSession
spark = SparkSession.builder \
    .appName("KafkaSensorStream") \
    .getOrCreate()

# Read from Kafka
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "100.xxx.xxx.xxx:9092") \ # Use your Kafka machine's Tailscale IP
    .option("subscribe", "sensor-data") \
    .load()

# Parse JSON from Kafka 'value' column
json_df = df.selectExpr("CAST(value AS STRING)") \
    .select(from_json(col("value"), schema).alias("data")) \
    .select("data.*")

# Output to console (can later change to DB, files, etc.)
query = json_df.writeStream \
    .outputMode("append") \
    .format("console") \
    .start()

query.awaitTermination()
```

Fig 6.2 processing code

○ Out

```
{"temperature": 25.1, "humidity": 40.3}
```

Fig 6.3 output

Bibliography

- **Apache Kafka Documentation** – <https://kafka.apache.org/documentation/>
- **Apache Spark Documentation** – <https://spark.apache.org/docs/latest/>
- **PySpark API Documentation** – <https://spark.apache.org/docs/latest/api/python/>
- **WSL Installation Guide** – <https://learn.microsoft.com/en-us/windows/wsl/>
- **Real-Time Data Streaming Concepts** – <https://www.confluent.io/what-is-real-time-streaming/>